What Does a Compression Transformer Actually Learn?

Dan Vu

Abstract

Lossless data compression reduces to sequential probability estimation. A transformer plugged into arithmetic coding is a compressor, and its compression quality is exactly its prediction quality. This paper asks what a small transformer learns when trained to compress binary sequences. We compare three models against the Laplace estimator and Context Tree Weighting (CTW) across IID sources with different Beta priors and a binary symmetric Markov chain. The results break down transformer performance into a prior encoded in the weights from offline training, and in-context adaptation from reading the test sequence. The prior is what beats classical universal coders. In-context adaptation alone cannot beat CTW on structured sources. An attention head probe gives a mechanistic account: the Bayes-optimal estimate $(k+\alpha)/(t+\alpha+\beta)$ is linearly encoded in Layer 0 attention outputs, with R^2 up to 0.82 for strong priors, outperforming a raw k/t probe at every layer for every model.

I. Introduction

A. Why Compression Matters

Compression comes in two flavors. Lossy compression discards information the receiver cannot perceive or does not need. JPEG drops high-frequency detail and MP3 removes inaudible components. Lossless compression discards nothing. Every binary bit of the original file can be recovered exactly. Lossless compression is required wherever one wrong bit corrupts the output. Some examples include executable code, legal records, medical images, cryptographic keys. There is no tolerance for approximation in those domains.

It turns out that the problem of lossless compression reduces to modeling the source distribution. A compressor assigns short codes to common patterns and long codes to rare ones. The better the probability model matches the true distribution, the shorter the average code. At scale this compounds: streaming platforms compress billions of files and digital archives grow by terabytes per day. A marginal improvement in the probability model shortens every symbol in every file. In this paper, we ask what a small transformer learns when trained to be that probability model.

B. Entropy and Optimal Codes

We test with binary sequences because it is a simple alphabet that can produce very specialized source distributions, and all modern digital data can be represented in binary.

A source emits binary symbols $x_1, x_2, \ldots$ drawn from $\{0, 1\}$ from a distribution p. A code assigns a binary string to each symbol. The optimal codelength assigned to symbol i with probability p_i is:

$$l_i^* = -\log_2 p_i. \tag{1}$$

The expected optimal codelength is therefore

$$H(X) = -\sum_x p(x) \log_2 p(x), \tag{2}$$

the Shannon entropy [1]. No compression algorithm can do better than $H(X)$ on average.

ML 445 Final Project, Spring 2026. Department of Electrical Engineering, University of Hawaii at Mānoa. Advised by Prof. Anders Høst-Madsen.

C. Arithmetic Coding and the Model-Coder Separation

Arithmetic coding separates the model from the coder. Any sequential conditional estimator $\hat{p}(x_t \mid x_{1:t-1})$ plugs in directly. The total codelength is

$$L(x^l) = \sum_{t=1}^{l} -\log_2 \hat{p}(x_t \mid x_{1:t-1}). \tag{3}$$

Bits per symbol (BPS) is $L(x^l)/l$. When $\hat{p} = p_{\text{true}}$, BPS equals entropy. Every excess bit above entropy is redundancy from the estimator. Compression quality equals estimation quality. The transformer, Laplace, and CTW are all probability estimators, and every experiment in this paper reduces to: which estimator predicts the next bit most accurately?

D. Universal Source Coding

A universal coder requires no knowledge of the source distribution. The redundancy of a code with length function L for a source with parameter $\boldsymbol{\theta}$ and sequences of length l is

$$R_l(L, \boldsymbol{\theta}) = \frac{1}{l}\mathbb{E}_{\boldsymbol{\theta}}[L(X^l)] - H_{\boldsymbol{\theta}}(X). \tag{4}$$

The Laplace estimator minimizes this redundancy sequentially: at each position t it estimates

$$\hat{p}(x_{t+1} = 1 \mid x_{1:t}) = \frac{k+1}{n+2}, \tag{5}$$

where k is the count of ones seen and $n = t-1$ is the total symbols seen. CTW [2] blends Laplace estimates over all context lengths up to depth d, and is asymptotically more powerful than Laplace on sources with memory. Both require no offline training. To beat them, a learned model must exploit knowledge of the source distribution acquired through prior training.

The transformer has two things working for it. One is a pretrain-learned prior: the distribution over sources that training encodes in the weights before any test sequence arrives. The other is in-context adaptation as the transformer reads the test sequence step by step. These contribute differently. On IID sources, a general transformer trained across all source parameters converges to the Laplace estimator, because a uniform mixture over p gives it exactly the prior Laplace assumes. A specialized transformer trained on one source family beats Laplace at short sequences because its prior is informative. On the binary symmetric Markov chain, the general transformer's in-context gains over Laplace are real but modest. CTW, which requires no offline training, catches up and beats it at longer sequences. Specialized training is what beats CTW. A linear probe on Layer 0 attention outputs recovers the Bayes-optimal estimate $(k+\alpha)/(t+\alpha+\beta)$ with R^2 up to 0.82 for strong priors. The prior correction is in the attention output, not the MLP. That is the mechanism.

II. Background

A. Arithmetic Coding and BPS

Arithmetic coding maintains an interval $[l, h) \subset [0, 1)$ and narrows it by $\hat{p}(x_t \mid x_{1:t-1})$ at each symbol. The final interval encodes the sequence, and its width determines the codelength via equation (3). BPS is the codelength divided by sequence length. The entropy $H(X) = -\sum_x p(x) \log_2 p(x)$ is the theoretical minimum BPS; no lossless code can do better on average. Every experiment in this paper reduces to: which estimator predicts the next bit most accurately?

B. The Laplace Estimator

The Laplace estimator is a parameter-free sequential predictor. At each position t, after seeing k ones in $t-1$ total symbols, it predicts

$$\hat{p}(x_t = 1 \mid x_{1:t-1}) = \frac{k+1}{t+2}. \tag{6}$$

This is Bayes-optimal under a uniform (Beta(1,1)) prior on the source parameter p. It requires only running counts and no training. Its redundancy decays as roughly $\frac{1}{2l}\log l$ bits per symbol on IID binary sources [1].

C. CTW

Context Tree Weighting (CTW) [2] extends Laplace to sources with memory. It maintains a binary tree of depth d, where each leaf holds a Laplace estimate conditioned on the corresponding context suffix. The tree blends these estimates up to the root, producing a prediction that adapts to the source's conditional structure. Like Laplace, CTW requires no offline training. At short sequences, CTW has no full context window and codes with flat priors, so it starts worse than Laplace. By moderate sequence lengths it catches up, and on sources with strong memory it eventually leads.

One implementation detail matters: CTW must call `get_distribution()` before calling `update()`. In the original notebook implementation, these were reversed, giving CTW access to the current symbol before predicting it. We fixed this by shifting the probability array by one position, ensuring the prediction at position t uses only symbols $x_1, \ldots, x_{t-1}$.

D. The Transformer Architecture

The transformer is minGPT-nano [3]: three transformer blocks, three causal self-attention heads per block, embedding dimension $d = 48$, and a context length of 499 tokens. The vocabulary is binary. The causal mask ensures position t attends only to positions $1, \ldots, t$, so no future symbol is visible. One forward pass produces all l conditional probabilities simultaneously. Training uses AdamW at learning rate 10^{-4}, batch size 64, 1000 iterations, on Apple MPS hardware. The total parameter count is approximately 0.11M.

III. Experiments and Results

All experiments use BPS averaged over 100 trials. p denotes the Bernoulli bias, l the test sequence length. Error bars are one standard error.

A. Experiment A: The General Transformer Learns the Laplace Estimator

A "general" transformer trains on sequences whose bias p is drawn uniformly from $(0, 1)$ at each training example. It never sees a fixed p twice.

The general transformer tracks Laplace at every sequence length and every p. The gap is small at short sequences and essentially zero by $l = 499$.

The match is the finding. Laplace is Bayes-optimal under a uniform prior on p. Training on a uniform mixture over p creates exactly that prior in the transformer's weights. The transformer has learned the Bayesian inference rule for this prior from data alone.

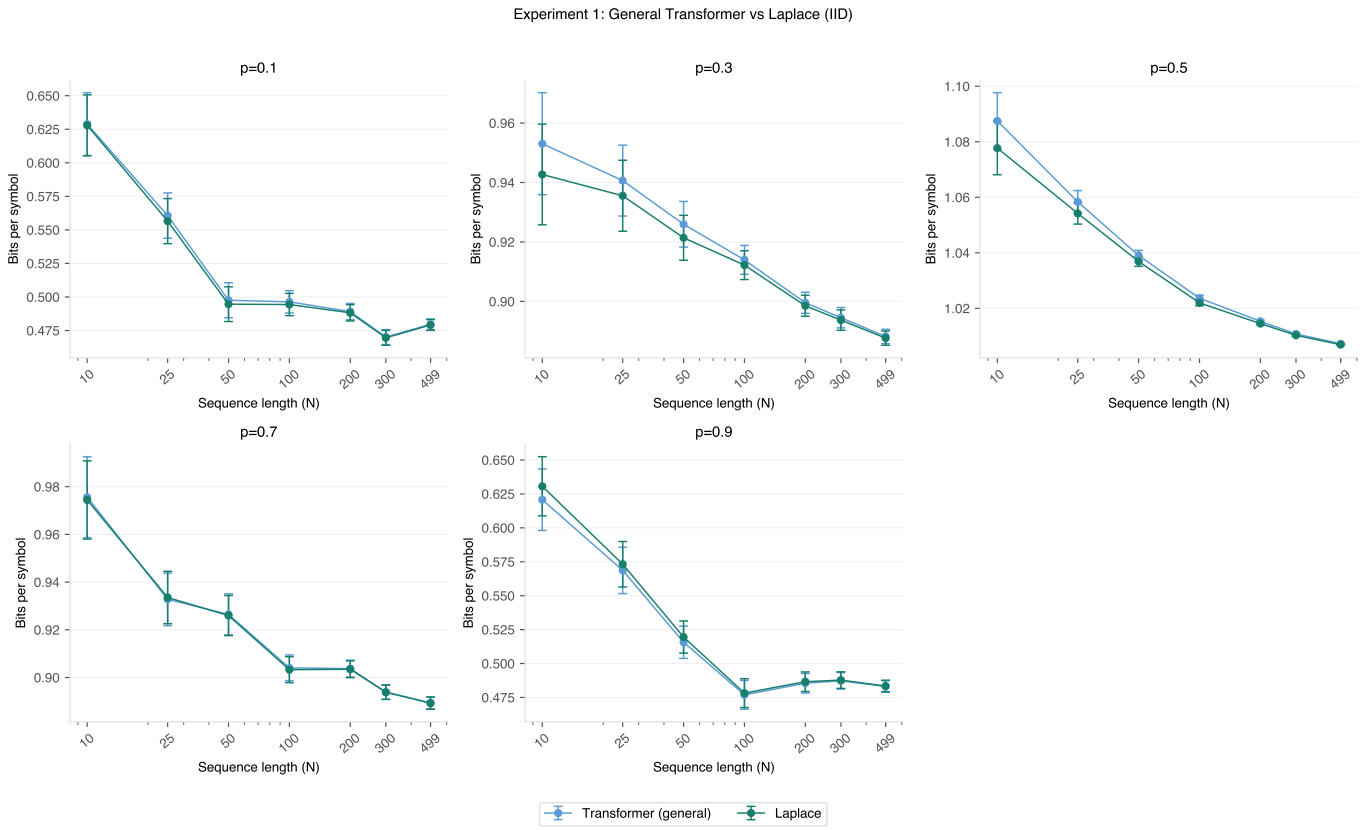

Figure 1. BPS vs. sequence length for the general transformer (blue) and Laplace (orange) at five values of p across $(0, 1)$. Entropy floors are shown as dashed lines. Error bars are one standard error over 100 trials.

B. Experiment B: A Specialized Transformer Beats Laplace at Short Sequences

A specialized transformer trains on sequences from $p = 0.3$ only. Concentrating training on one source gives the model a strong implicit prior for that value.

The specialized transformer leads Laplace by a meaningful margin at short sequences. The gap collapses as the sequence grows.

This makes sense because the prior is most valuable when there is little data. At $l = 10$ the empirical count k/t is noisy, so the pretrain-learned prior that already knows $p \approx 0.3$ dominates. As l grows, the in-context component takes over. Both the transformer and Laplace converge toward the true parameter, and the prior advantage shrinks. The $p = 0.7$ result is identical by symmetry: $H(0.3) = H(0.7)$ because the binary entropy function is symmetric around $p = 0.5$.

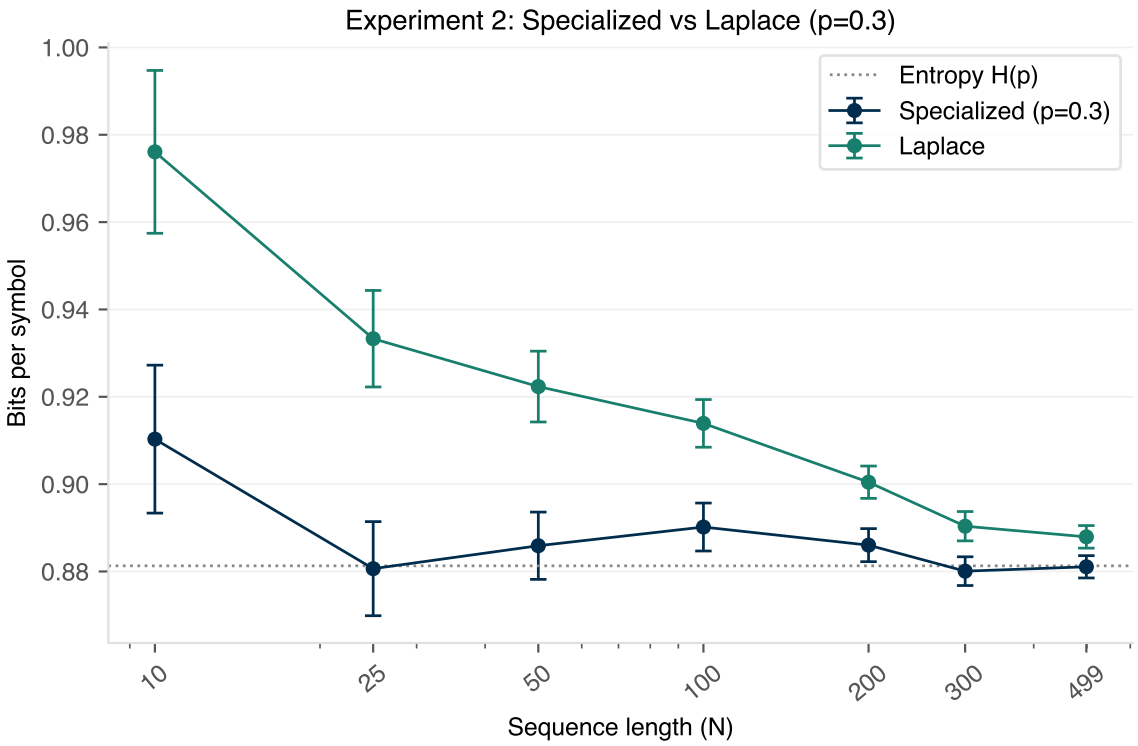

Figure 2. BPS vs. sequence length for the specialized transformer (dark blue), general transformer (light blue), Laplace (orange), and entropy (dashed) at $p = 0.3$.

C. Experiment C: Beta Prior Generalization

Five transformers train under five Beta priors: Beta(1,1), Beta(0.5,0.5), Beta(5,5), Beta(5,2), and Beta(2,5). Each is evaluated against Laplace and a Bayes-optimal estimator that uses the true prior analytically. The question is whether the prior advantage from Experiment B is specific to one fixed p, or whether it generalizes across source families.

Under Beta(1,1) the transformer, Laplace, and optimal estimator all align, consistent with Experiment A. Under concentrated priors like Beta(5,5) and Beta(0.5,0.5), the transformer sits close to the optimal estimator and clearly above Laplace at short sequences.

This is the strongest version of the prior argument. Trained under any Beta prior, the transformer tracks the Bayes-optimal estimator for that prior across all five families tested. It is learning the inference rule for the prior family, not memorizing a fixed bias. The gap to optimal is positive but small: the transformer approximates the Bayesian computation rather than performing it exactly.

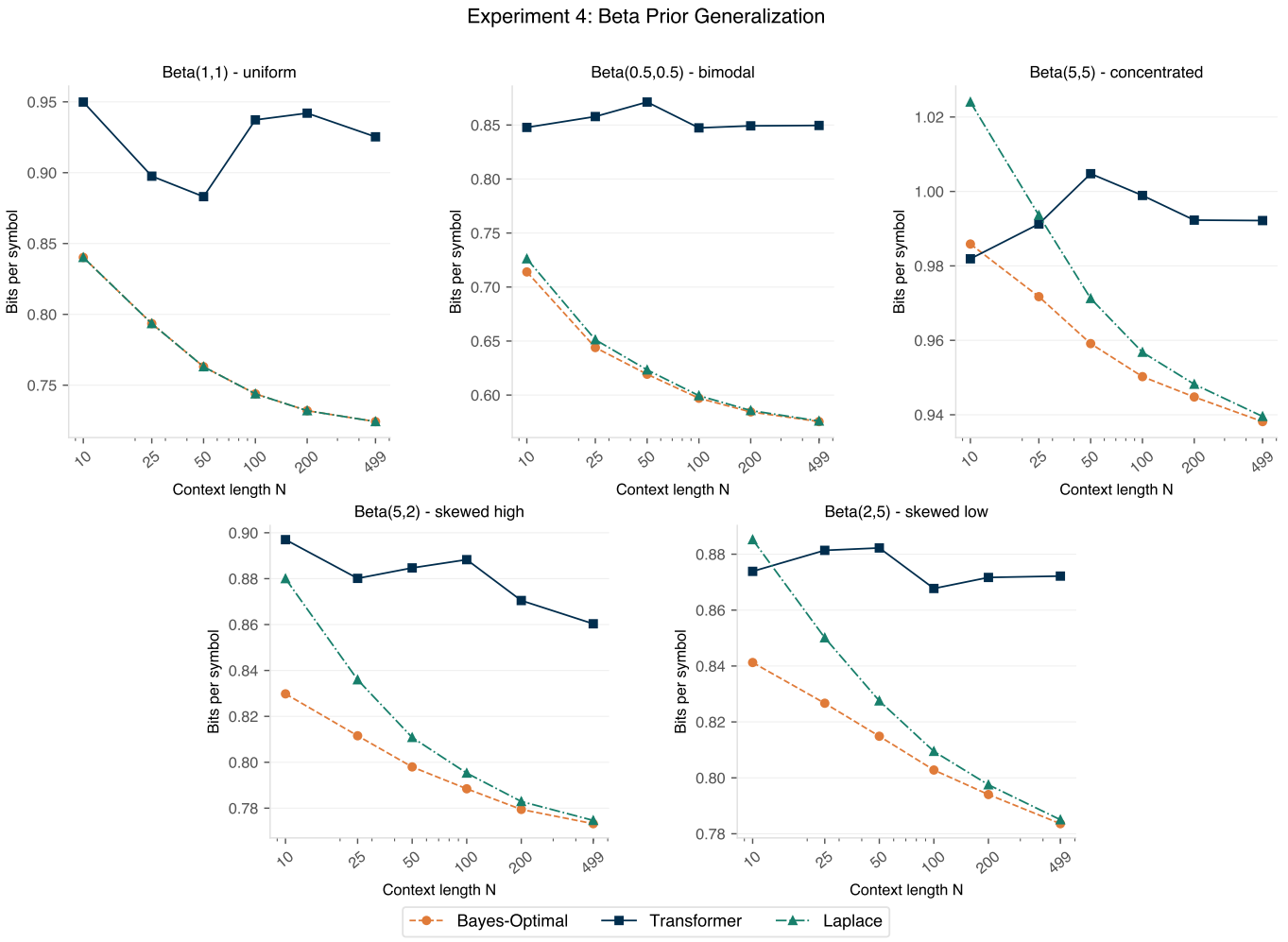

Figure 3. BPS vs. sequence length for the transformer (blue), Laplace (orange), CTW (green), and the Bayes-optimal estimator (red) under each Beta prior. Entropy floors are dashed.

D. Experiment D: Binary Symmetric Markov Chain

This experiment moves to a source with memory. The binary symmetric Markov chain has a single parameter p_{stay}: the probability of staying in the current state. We compare four estimators: a specialized transformer trained per p_{stay}, the general IID transformer from Experiment A, Laplace, and CTW at depth 8.

At $p_{\text{stay}} = 0.5$ the source is memoryless and all four estimators align. The gap opens at high p_{stay}, where the source has strong temporal structure. CTW starts poorly at short sequences because its depth-8 tree has seen no useful context yet, but it recovers and pulls ahead of Laplace by moderate lengths.

The IID transformer beats Laplace at short sequences, but CTW, with no offline training, catches and passes it by moderate lengths. On a source with memory, in-context adaptation alone is not enough.

The specialized transformer is a different story. Trained on the same p_{stay}, it knows the transition structure before the first test symbol arrives. It leads all three other estimators at every sequence length, and the gap holds as l grows.

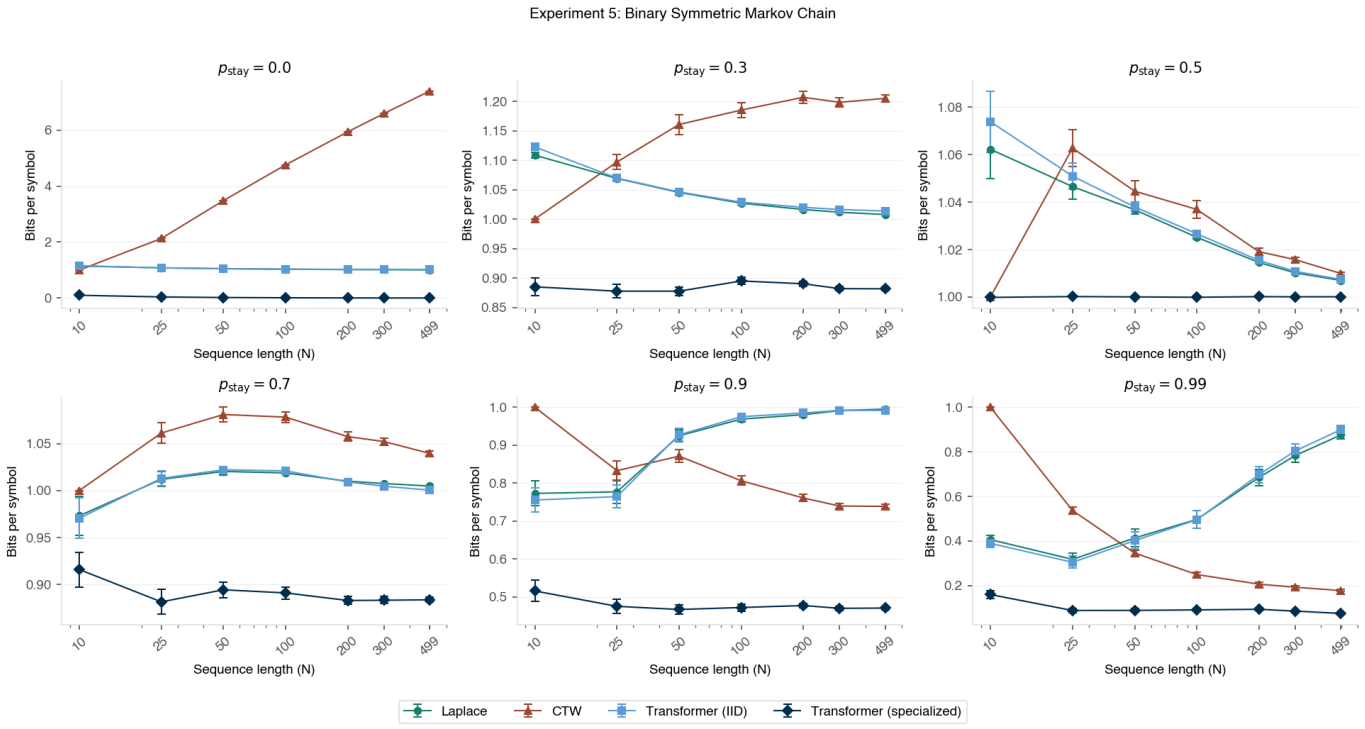

Figure 4. BPS vs. sequence length on the binary symmetric Markov chain for four estimators across six values of p_{stay}.

E. Experiment E: Attention Head Probe

The behavioral experiments show the transformer encodes a prior. This experiment asks where that prior lives mechanistically.

We fit Ridge regression probes from this 16-dim vector to two targets at each position t:

$$\text{empirical:} \quad k_t/t \tag{7}$$

$$\text{Bayes-optimal:} \quad (k_t + \alpha)/(t + \alpha + \beta) \tag{8}$$

where k_t is the running count of ones. All five Beta-prior models from Experiment C are probed on 50 fixed test sequences ($p = 0.5$, length 499). R^2 is reported per layer per head, then averaged over heads.

The Bayes probe beats the k/t probe at every layer for every model. The prior correction (α, β) is present at the attention output, not deferred to the MLP downstream.

Layer 0 dominates for four of the five models. The counting signal is strongest at the first attention layer and degrades through Layers 1 and 2. This is consistent with a picture where Layer 0 computes the estimate and later layers transform it toward the final output.

Beta(5,5) is the exception. It is the only model where the R^2 stays high across all three layers. The symmetric strong prior produces a counting representation that persists rather than being consumed.

R^2 scales with prior concentration. Beta(0.5,0.5) peaks at roughly 0.43 Bayes R^2; Beta(5,2) reaches 0.82. Beta(0.5,0.5) is a U-shaped prior that concentrates mass near 0 and 1, so the model spends less of its capacity tracking the running count and more on the extreme-probability regime.

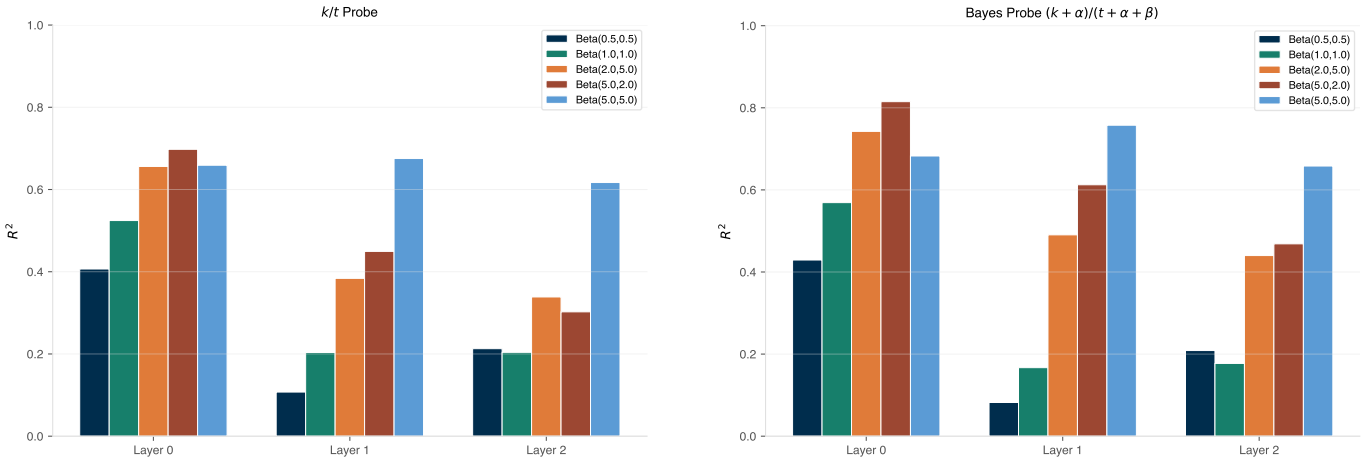

Figure 5. Mean R^2 of a linear probe from attention head outputs averaged over 3 heads per layer. Left: k_t/t probe. Right: Bayes-optimal $(k_t + \alpha)/(t + \alpha + \beta)$ probe. All five Beta-prior models shown.

IV. Discussion

The transformer holds two things at once: a frozen prior from offline training and an in-context estimator that reads the test sequence live. The behavioral results show which one matters.

The prior determines whether the transformer beats classical universal coders. The general transformer has a uniform prior and converges to Laplace. The specialized transformers have informative priors and beat Laplace at short sequences. On the Markov chain, the specialized transformer leads CTW at every sequence length because it already knows the transition structure before the first test symbol arrives. The IID transformer's in-context learning cannot replicate this. CTW's online reconstruction catches up and surpasses it at longer sequences. On sources where the structure is known in advance, the prior advantage persists as l grows.

The probe experiment localizes where that prior lives. Layer 0 attention outputs linearly encode the Bayes-optimal estimate for the training prior, not the raw empirical count. The prior correction (α, β) is in the value projection or attention weight structure, already applied before the output projection runs. Later layers transform the representation but do not recompute it from scratch, except in Beta(5,5) where the counting direction persists across all three layers.

V. What Next

The Markov experiment showed the IID-trained transformer exploits autocorrelation it was never trained on. The natural question is whether the same counting head activates on Markov sequences. If the Bayes probe R^2 holds on Markov test sequences, counting is a general primitive the model learned from IID data and applies broadly. If it collapses, the counting direction is IID-specific and the in-context gains on Markov sources come from a different mechanism entirely.

The probe was run on a fixed model snapshot. Running it on checkpoints at 100, 250, 500, and 1000 training iterations would show whether counting emerges early and sharpens gradually, or appears suddenly past some threshold. That trajectory would connect the mechanistic picture to the behavioral one: the iteration at which R^2 rises should match the iteration at which BPS drops below Laplace.

VI. Supplemental

A. Data

All data is synthetic. Binary sequences were generated programmatically from the specified source distributions (Bernoulli IID, Beta-prior IID, binary symmetric Markov chain). No real-world datasets were used.

B. AI Usage

I used AI to discuss ML and information theory concepts. To do this, I pasted excerpts from "Elements of Information Theory" by Cover and Thomas and had LLMs such as ChatGPT and Claude explain back the book. Initially, when the project first started and the first lines of code were written, we only used AI to generate syntax patterns (Stack Overflow Style). This was done to engage with information theory concepts, as a substitute for homework problems. However, when experiments were run, we used Claude to design experiments using text, and then asked it to generate code blocks for pasting into Python notebooks. No agentic workflows were used to perform experiments. However, Claude code was used to compile results and place them into the LaTex file. Claude code also wrote some prose for the experiments. Claude was also used to develop wording and to tighten up hand crafted sentences. A final pass using Claude was done to proofread the document. Screenshots of the LLM assissted workflow can be found in the project folder.

REFERENCES

[1] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Hoboken, NJ: Wiley-Interscience, 2006.
[2] G. Schamberg, "Context tree weighting in Python," https://github.com/gabeschamberg/context-tree-weighting, 2020, gitHub repository.
[3] A. Karpathy, "minGPT: A minimal PyTorch re-implementation of the GPT training," https://github.com/karpathy/minGPT, 2020, gitHub repository.
[4] A. Høst-Madsen, M. Zaeri Amirani, and N. P. Santhanam, "A bound for learning lossless source coding with online learning," in *IEEE Int. Symp. Inf. Theory (ISIT)*, Taipei, Taiwan, Nov. 2024.